PROJECT DOCUMENTATION REPORT

AI-POWERED CUSTOMER SUPPORT TICKETING SYSTEM

Intel Case Flow — A Salesforce-style, AI-assisted Telecom Support Platform

Project Title	AI Powered Customer Support Ticketing System (Intel Case Flow)
Repository	github.com/Venkatakeerthi13/intel-case-flow
Owner	Venkatakeerthi13
Primary Stack	React 19 + TypeScript, TanStack Start/Router, Tailwind CSS 4, Supabase
Built With	Lovable (AI application builder / Lovable Cloud)
Document Type	Final Project Report

This document has been prepared following the standard Project Report Format (Introduction → Ideation → Requirement Analysis → Design → Planning → Testing → Results → Advantages/Disadvantages → Conclusion → Future Scope → Appendix), and is generated directly from the project's public source repository so that every technical claim traces back to actual code, schema, and configuration in the repository.

Table of Contents

1. Introduction

1.1 Project Overview

Intel Case Flow is a web-based, AI-assisted customer support ticketing system built for a telecommunications support organisation. It follows a Salesforce Service-Cloud style layout — a case-centric workspace where support agents track customer issues ("cases") from creation through resolution, alongside supporting records for Accounts, Contacts, Support Categories and SLA Policies. The application was developed using **Lovable**, an AI-assisted application builder, and is backed by a **Supabase** (PostgreSQL) database for data storage, authentication and row-level security.

The distinguishing feature of the system is its built-in **AI Feedback** module: agents can request an AI-generated draft reply for any case (via an LLM call routed through the Lovable AI Gateway) and then log whether that suggestion was Accepted, Modified, or Rejected — creating a feedback loop that can be used to evaluate and improve AI-assisted response quality over time.

1.2 Purpose

Traditional telecom support desks handle a high volume of repetitive cases — network outages, billing disputes, SIM activation, roaming charges — that consume agent time on manual case logging, prioritisation and reply drafting. The purpose of this project is to:

- Centralise case, account, contact, category and SLA data in a single, structured application instead of spreadsheets or disconnected tools.
- Automate repetitive back-office logic (case numbering, priority defaulting, SLA due-date calculation, round-robin agent assignment, resolution-time capture) inside the database itself, so it is consistent regardless of which client calls it.
- Reduce first-response time by letting agents generate a professional, empathetic, on-brand draft reply with a single click, using an LLM that is given full case context (subject, description, issue type, priority, customer and SLA data).
- Give support managers visibility into operational health through a dashboard and analytics/reports view (open vs. closed cases, SLA compliance, case volume trends).
- Provide a foundation for continuously measuring and improving AI-suggested replies through the AI Feedback log.

2. Ideation Phase

2.1 Problem Statement

“How might we help telecom support agents resolve customer cases faster and more consistently, without adding manual administrative overhead, while giving managers real-time visibility into SLA performance?”

Support teams at telecom providers deal with high case volumes across very different issue types — network outages, slow internet, billing disputes, SIM activation, roaming charges and device faults — each with a different urgency profile. Without a structured system, agents spend time manually assigning priority, calculating SLA deadlines, picking who should work a case, and composing replies from scratch for each new ticket. This slows first response, causes inconsistent tone/quality across agents, and makes it hard for managers to see whether SLAs are being met.

2.2 Empathy Map Canvas

The core personas considered during ideation were the **Support Agent** (day-to-day case handler) and the **Support Manager** (oversees SLA and team performance).

Quadrant	Support Agent	Support Manager
Says	“I need the case history and customer details in one screen before I reply.”	“Are we hitting our SLA commitments this week?”
Thinks	“This issue type is common — I write nearly the same reply every time.”	“Which categories generate the most escalations?”
Does	Reads the case, checks account/contact info, drafts a reply, updates status.	Reviews open/closed counts, SLA compliance %, and categ
Feels	Pressured by SLA countdowns; repetitive replies feel inefficient.	Concerned about visibility gaps and inconsistent response quality a

2.3 Brainstorming

Ideas generated and evaluated before converging on the final feature set:

- A single case-management workspace with a defined status lifecycle (New → In Progress → Escalated → Closed).
- Auto-generated, human-readable case numbers (e.g. CS-01001) instead of raw UUIDs.
- Category-driven default priority so new cases don't start unclassified.
- Database-enforced SLA due-date calculation tied to a configurable SLA Policy table (per priority level).
- Automatic, load-balanced agent assignment (round-robin over active Support Agents by open case count) to avoid manual triage.
- An AI-assisted reply generator using case context, reducing the time an agent spends drafting a first response.
- An AI Feedback log so the accuracy/usefulness of AI drafts can be tracked (Accepted / Modified / Rejected) rather than trusted blindly.
- A Reports & Analytics view for SLA compliance and case-volume insight, so ideas that only benefited agents but not managers were deprioritised in favour of this shared view.

3. Requirement Analysis

3.1 Customer Journey Map

End-to-end journey of a case from the customer's issue to closure:

Stage	What Happens
1. Issue occurs	Customer experiences a network, billing, SIM, roaming or device issue and contacts support.
2. Case logged	An agent creates a case (subject, description, issue type, account/contact) via the New Case dialog.
3. Auto-triage	A database trigger assigns a case number, defaults the priority from the category, calculates the SLA du
4. Investigation	The agent opens the case detail page, reviews account, contact, category, priority and SLA-due informat
5. AI-assisted reply	The agent optionally triggers “Generate AI Reply”; the system calls an LLM with full case context and ret
6. Feedback logged	The agent marks the AI suggestion as Accepted, Modified or Rejected, with optional comments — creati
7. Resolution	The agent updates status through In Progress / Escalated to Closed; on closure the system stamps clos
8. Reporting	Managers review the Dashboard and Reports pages for SLA compliance, open/closed counts, and categ

3.2 Solution Requirement

Functional Requirements

- Create, view, update and track support Cases with status, priority, issue type, category, account, contact and assigned agent.
- Maintain master data for Accounts, Contacts, Support Categories, SLA Policies and Agents.
- Auto-generate case numbers and AI-feedback numbers.
- Auto-calculate SLA due dates and resolution time based on configurable SLA policies.
- Auto-assign new cases to an active Support Agent using a load-balanced (least open-case-count) rule.
- Generate an AI-drafted customer reply from case context, editable and savable to the case record.
- Log agent feedback on each AI suggestion (Accepted / Modified / Rejected) with comments.
- Provide a Dashboard summarising open cases, closed cases and SLA compliance.
- Provide a Reports view for case analytics (status, priority, SLA-due and category breakdowns).
- Secure all API access with Supabase authentication (Bearer/JWT) for server-side data operations.

Non-Functional Requirements

- Usability — a clean, dashboard-style UI consistent with familiar CRM/ticketing tools (built with shadcn/ui + Radix primitives).
- Performance — client-side data fetching/caching via TanStack Query to minimise redundant network calls.
- Reliability — business rules (numbering, SLA calculation, assignment, timestamps) enforced at the database layer via triggers, not just in the UI, so they hold regardless of client.
- Security — Row Level Security (RLS) enabled on every table; server functions validate Bearer tokens before performing privileged operations.
- Maintainability — fully typed TypeScript codebase with a generated Supabase Database type and Zod-validated server function inputs.
- Scalability — stateless server functions (TanStack Start server functions) and a managed Postgres backend (Supabase) that can scale independently of the frontend.

3.3 Data Flow Diagram

Level-1 data flow through the system:

```
[Customer]
| reports issue
v
[Support Agent] --(1) create/update case--> [React UI: Cases, Accounts,
^ Contacts, Categories, SLA]
| |
| TanStack Query + Supabase JS
| v
| [Supabase (PostgreSQL) API]
| |
| triggers: set_case_defaults / handle_case_update
| (case #, priority, SLA due date, auto-assign,
| closed_at + resolution_time_hours)
| v
| [cases, accounts, contacts,
| support_categories, sla_policies,
| agents, ai_feedback tables]
| |
|<--(2) AI reply request-- [Case Detail Page] -+
| | createServerFn: generateCaseReply
| v
| [TanStack Start server fn] --> [Lovable AI Gateway] --> [LLM: Gemini]
| | draft reply text
| v
| saved to cases.ai_suggested_response
| |
+<--(3) Accept/Modify/Reject feedback --> [ai_feedback table]

[Manager] --(4) views--> [Dashboard / Reports] <--aggregated queries-- [cases,
sla_policies]
```

3.4 Technology Stack

Layer	Technology	Role in the Project
UI Framework	React 19 + TypeScript	Component-based, type-safe frontend
Routing / SSR	TanStack Start + TanStack Router	File-based routing, server functions, SSR
Data Fetching	TanStack Query (@tanstack/react-query)	Client-side caching of Supabase queries
Styling	Tailwind CSS 4	Utility-first styling
Component Library	shadcn/ui on Radix UI primitives	Accessible dialogs, dropdowns, tables, forms, sidebar, charts, etc.
Forms & Validation	React Hook Form + Zod	Typed form state and schema validation (client and server)
Charts	Recharts	Reports & Analytics visualisations
Backend / DB	Supabase (PostgreSQL)	Managed Postgres, Auth, Row Level Security, auto-generated RES
AI Integration	Vercel “ai” SDK + @ai-sdk/openai-compatible via Lovable AI Gateway	generateText() calls google/gemini-3-flash-preview for c
Build Tool	Vite 8	Dev server and production bundling
Package Manager	Bun	Dependency management and lockfile (bun.lock)
Tooling	ESLint 9 + typescript-eslint, Prettier	Linting and formatting
Platform	Lovable / Lovable Cloud	AI-assisted app scaffolding, hosting, connected Supabase project

4. Project Design

4.1 Problem – Solution Fit

Problem	Solution Provided
Manual, inconsistent case prioritisation and SLA tracking	Trigger-based auto-assignment of priority (from category) and SLA due date (fi
Uneven agent workload from manual case assignment	Least-open-case round-robin auto-assignment among active Support Agents
Slow, inconsistent first-response drafting	One-click AI-generated, context-aware draft reply (case number, subject, issue
No way to know if AI drafts are actually useful	AI Feedback table capturing Accepted / Modified / Rejected + agent comments
No unified visibility for managers	Dashboard (open/closed cases, SLA compliance) and a dedicated Reports pag
Scattered account/contact/category/SLA reference data	Dedicated Accounts, Contacts, Categories and SLA Policies pages backed by

4.2 Proposed Solution

Intel Case Flow is proposed as a single-page application with a persistent app shell (sidebar navigation) and the following functional routes:

Route	Purpose
/ (Dashboard)	Summary of open cases, closed cases and SLA compliance
/cases	List/filter all cases; entry point to create a new case
/cases/$caseId	Case detail: full record, status/priority editing, AI reply generation, feedback logging
/accounts	Manage customer Accounts (with contact and case counts)
/contacts	Manage Contacts linked to Accounts
/categories	Manage Support Categories and their default priority
/sla-policies	Manage SLA Policies (resolution hours, escalation flag per priority)
/ai-feedback	Review logged AI Feedback entries across cases
/reports	Analytics: case volume, status/priority mix, SLA performance

Business rules that would otherwise live only in application code (case numbering, priority/SLA defaulting, agent assignment, resolution-time capture) are implemented as PostgreSQL trigger functions, so the rules apply consistently no matter which client writes to the database.

4.3 Solution Architecture

```
+------------------------------+
| React 19 + TS UI |
| (TanStack Router file routes, |
| shadcn/ui + Tailwind CSS) |
+---------------+-----------------+
|
TanStack Query (client-side caching)
|
+----------------------------+-----------------------------+
| |
Supabase JS client (browser) TanStack Start server functions
src/integrations/supabase/client.ts src/lib/ai.functions.ts
(public anon key, RLS-scoped) + auth-middleware.ts (Bearer JWT check)
| |
v v
+---------------------------+ +---------------------------------+
| Supabase (PostgreSQL) | | Lovable AI Gateway (LOVABLE_API_KEY)
| Tables: agents, accounts, | | -> generateText() -> Gemini model |
| contacts, support_ | +---------------------------------+
| categories, sla_policies, |
| cases, ai_feedback |
| + triggers/functions for |
| numbering, SLA, assignment |
| + Row Level Security |
+---------------------------+
```

The frontend never talks to the LLM directly: the “Generate AI Reply” action calls a TanStack Start server function (`generateCaseReply`), which runs on the server, reads the `LOVABLE_API_KEY` secret, and calls the Lovable AI Gateway using the Vercel `ai` SDK's `generateText()` with model `google/gemini-3-flash-preview`. This keeps the API key off the client and lets the system prompt enforce tone, length and formatting rules for every generated reply.

5. Project Planning & Scheduling

5.1 Project Planning

The project was structured into the following work packages, aligned to the layers of the architecture:

Phase	Key Activities	Output
1. Ideation & Requirements	Problem statement, empathy mapping, brainstorming, functional & non-functional requirement listing	Section 2 & 3 of this document
2. Database Design	Define agents, accounts, contacts, support_categories, sla_policies, cases and ai_feedback tables; RL	supabase/migrations/*.sql
3. Frontend Scaffolding	TanStack Start/Router routes, app shell/sidebar, shadcn/ui components, Tailwind theme	src/routes/*, src/components/*
4. Core CRUD Features	Cases list/detail, Accounts, Contacts, Categories, SLA Policies pages wired to Supabase via queries.ts	src/lib/queries.ts + route pages
5. AI Integration	Server function for AI reply generation, AI Gateway wiring, AI Feedback logging UI/table	src/lib/ai.functions.ts, src/lib/ai-gateway.server.ts, ai_
6. Analytics	Dashboard summary cards; Reports page with charts (Recharts)	src/routes/index.tsx, src/routes/reports.tsx
7. Testing & Hardening	Type-checking, linting, functional walkthrough of the case lifecycle, RLS/auth verification	eslint.config.js, manual functional test pass
8. Deployment	Build via Vite, hosted through Lovable Cloud, connected Supabase project	vite.config.ts, supabase/config.toml

6. Functional and Performance Testing

6.1 Performance Testing

Representative functional test scenarios covering the case lifecycle and AI feature:

#	Scenario	Expected Result
TC-01	Create a new case without setting priority, but with a category selected	Priority auto-filled from the category's default_priority; case_number auto-ge
TC-02	Create a case with priority = Critical	SLA due date = created_at + 4 hours (per Critical Response Policy)
TC-03	Create a new case with no agent specified	Case is auto-assigned to the active Support Agent with the fewest open ca
TC-04	Change a case's status to Closed	closed_at is stamped and resolution_time_hours is computed automatically
TC-05	Re-open a previously closed case (status changed away from Closed)	closed_at and resolution_time_hours are cleared
TC-06	Change a case's priority after creation	sla_due_date is recalculated from the new priority's SLA policy
TC-07	Click “Generate AI Reply” on a case with full context	A draft reply under ~180 words is returned, addressing the customer, refer
TC-08	Trigger AI generation with LOVABLE_API_KEY unset	Server function throws “AI is not configured for this project” and the UI surfa
TC-09	Log feedback as Modified with a comment	A new ai_feedback row is created with an auto-generated feedback_numb
TC-10	Call a protected server function without a Bearer token	Request is rejected with “Unauthorized: No authorization header provided”
TC-11	Load the Dashboard and Reports pages	Open/closed counts and SLA compliance figures match the underlying cas

Performance considerations: list views (Cases, Accounts, Contacts) are fetched through TanStack Query, which caches results client-side and avoids redundant round-trips on repeated navigation. Because SLA due-date calculation, priority defaulting and agent assignment are implemented as PostgreSQL trigger functions rather than application-layer loops, these operations execute in a single database round trip per write, independent of UI performance. The AI reply path is the primary latency-sensitive operation, since it depends on an external LLM call (google/gemini-3-flash-preview) via the Lovable AI Gateway; the UI shows a loading/failure state around this call so it does not block the rest of the case workspace.

7. Results

7.1 Output Screenshots

This report is generated directly from the source repository rather than a running deployment, so live screenshots are not embedded here. The screens produced by the implemented routes are summarised below — insert captured screenshots of the deployed application in the corresponding placeholders before final submission.

Screen	What It Shows
Dashboard (/)	Open Cases, Closed Cases and SLA Compliance summary cards
Cases List (/cases)	Filterable table of all cases with status, priority and SLA-due indicators
Case Detail (/cases/$caseId)	Full case record, status/priority controls, “Generate AI Reply” action, feedback logging
Accounts (/accounts)	Account list with contact and case counts per account
Contacts (/contacts)	Contact list linked to accounts
Categories (/categories)	Support category list with default priority
SLA Policies (/sla-policies)	Configured SLA policies per priority level
AI Feedback (/ai-feedback)	Log of AI suggestions and their Accepted/Modified/Rejected outcomes
Reports (/reports)	Charted analytics on case volume, status/priority mix, and SLA performance

[Screenshot placeholder — Dashboard]

[Screenshot placeholder — Case Detail with AI Reply]

[Screenshot placeholder — Reports & Analytics]

8. Advantages & Disadvantages

Advantages	Disadvantages / Limitations
Business rules (numbering, SLA, assignment) enforced at the database level, guaranteeing consistency across any client.	Row Level Security policies currently allow public/anonymous read/write acc
Single-click AI reply generation reduces first-response time and standardises tone.	AI reply quality and correctness depend entirely on the case description age
AI Feedback log enables measurable tracking of AI suggestion quality over time (Accepted/Modified/Rejected).	Feedback is currently free-text/status only — there is no automated quality s
Modern, type-safe stack (React 19, TypeScript, Zod, generated Supabase types) reduces runtime errors.	The AI reply path introduces an external dependency (Lovable AI Gateway /
Round-robin, load-aware agent assignment removes manual triage overhead.	Assignment only considers current open-case count, not agent skill/specialis
Reports/Dashboard give managers real-time SLA and volume visibility without a separate BI tool.	Analytics are computed from live case data with no historical snapshotting/tr

9. Conclusion

Intel Case Flow demonstrates a practical, production-shaped approach to combining a conventional CRM-style ticketing workflow with an AI-assisted response layer. By pushing case numbering, prioritisation, SLA calculation and agent assignment into database trigger functions, the system keeps its core business logic consistent and auditable regardless of which client touches the data. Layering an LLM-backed reply generator on top — accessed only through a server-side function that validates auth and hides the API key — adds a genuinely useful productivity feature without compromising the integrity of the core ticketing model. The AI Feedback mechanism closes the loop by giving the team a structured way to judge whether the AI is actually helping, rather than assuming it is. Overall, the project meets its stated purpose: it gives telecom support agents a faster, more consistent way to work cases, and gives managers a clear view of SLA performance, while remaining a solid foundation to harden further for production (see Section 10).

10. Future Scope

- Replace the current “Public demo access” RLS policies with role-based policies tied to authenticated agent identities (agent, manager, admin roles).
- Add skill/specialisation-aware routing to case auto-assignment, not just open-case-count load balancing.
- Introduce automated SLA breach alerts/notifications (email, Slack, in-app) rather than dashboard-only visibility.
- Extend the AI Feedback loop into a lightweight evaluation pipeline — aggregate Accept/Modify/Reject rates per issue type to flag categories where AI drafts need prompt tuning.
- Support customer-facing self-service (case submission portal, status tracking) in addition to the agent-facing workspace.
- Add full-text/semantic case search and duplicate-case detection using the same AI Gateway integration.
- Historical reporting: snapshot daily/weekly case and SLA metrics into a reporting table for real trend analysis instead of live-only aggregation.
- Multi-channel case creation (email-to-case, chat widget) feeding the same cases table and automation triggers.

11. Appendix

Source Code

Key source files referenced in this document:

File	Purpose
supabase/migrations/*.sql	Full schema: agents, accounts, contacts, support_categories, sla_policies, cases, ai_feedb
src/lib/queries.ts	Typed query definitions and shared constants (CASE_STATUSES, CASE_PRIORITIES, IS
src/lib/ai.functions.ts	generateCaseReply server function — builds the LLM prompt from case context
src/lib/ai-gateway.server.ts	Lovable AI Gateway provider wiring for the “ai” SDK
src/integrations/supabase/client.ts	Browser Supabase client (anon key, RLS-scoped)
src/integrations/supabase/auth-middleware.ts	Server-side Bearer-token validation middleware for protected server functions
src/routes/*.tsx	Application pages: dashboard, cases, case detail, accounts, contacts, categories, sla-polic
package.json	Full dependency manifest (React 19, TanStack Start/Router/Query, Supabase JS, Tailwind

Dataset Link

No external/static dataset is used. All data is generated and stored live in the connected Supabase PostgreSQL project (project ref: `anysbxthzvqnomtsdcwe`, see `supabase/config.toml`), seeded initially with demo Agents, Accounts and reference data for SLA Policies and Support Categories (see `supabase/migrations/`).

GitHub & Project Demo Link

GitHub Repository: https://github.com/Venkatakeerthi13/intel-case-flow

Project Demo: the application is built and hosted via Lovable / Lovable Cloud, connected to the Supabase project referenced above. *(Insert the live Lovable preview/production URL here if one is being shared alongside this report.)*